AccountAgent: An End-to-End AI Accounting Assistant System with Multimodal Voucher Processing and Intelligent Financial Control

Yulu Huang[†], Niannian Yu, Yaxin Yang, Jinpeng Lv

Jiangxi University of Finance and Economics

[†]Corresponding author: yuluhuang324@gmail.com

Abstract

Enterprise accounting is a foundational yet labor-intensive function: vouchers are keyed in manually, business and financial data live in disconnected systems, and analysis arrives too late to inform proactive decisions. We present AccountAgent, an end-to-end AI accounting assistant that reconstructs the accounting workflow around a multimodal large language model and a set of deterministic financial engines. AccountAgent couples (i) a vision–language model that recognizes more than thirty receipt types—value-added-tax invoices, bank statements, expense-reimbursement forms, and long-tail documents—and extracts amounts, tax rates, and counterparties at above 99% accuracy; (ii) a knowledge-base-driven subject-matching and double-entry generation engine with hard balance validation; (iii) an event-driven business–finance integration layer in which the confirmation of a sales order or a purchase receipt posts the corresponding receivable or payable voucher immediately; (iv) a multidimensional analysis engine combining rule-based thresholds, moving averages, ordinary-least-squares trend forecasting, and 2σ anomaly detection; (v) a full-pipeline tax-compliance engine covering value-added tax, corporate income tax, and stamp duty with deadline monitoring and one-click declaration; and (vi) a fund-management module that forecasts 30-day cash positions and distills them into a health score with graded risk levels and actionable recommendations. We describe the five-layer cloud-native architecture, the core algorithms, and a complete open-source reference implementation in approximately 5,000 lines of Python, in which every module of the design is realized and executable. The system drives monthly error rates below 0.3%, compresses period close from days to one day, and shifts the accountant's role from mechanical entry toward analysis and decision support.

Keywords: AI Accounting; Multimodal Large Language Model; Voucher Processing; Business–Finance Integration; Anomaly Detection; Tax Compliance; Fund Forecasting; Cloud Native

1 Introduction

Accounting is the ledger of enterprise reality, and the integrity of that ledger underpins every downstream decision—from pricing and budgeting to investment and compliance. Yet the work of maintaining it—classifying vouchers, reconciling modules, and closing periods—remains disproportionately manual, slow, and error-prone. Traditional accounting workflows are constrained by three structural pain points:

- **Low efficiency of entry.** Vouchers are keyed in by hand from paper or scanned documents; the same information is re-typed across systems, and manual entry is itself the dominant source of bookkeeping error.
- **Disconnected business and financial data.** Sales, procurement, inventory, reimbursement, and fund

management run in separate systems, so the ledger is reconstructed periodically rather than updated as business events occur, and period close becomes a reconciliation exercise.

- **Data lag in analysis.** Analysis arrives after the period has ended, too late to warn of a deteriorating margin, an overdue receivable, or an impending cash shortfall while intervention is still possible.

Artificial intelligence and large language models (LLMs) offer a path beyond manual bookkeeping [Brown et al., 2020; Touvron et al., 2023; Dubey et al., 2024]. A multimodal large model can recognize vouchers across more than thirty receipt types, match accounting subjects against a domain knowledge base, and generate bookkeeping vouchers automatically, while integrated analysis engines support cash-flow forecasting, cost monitoring, and sales-trend analysis [Dosovitskiy et al., 2021; Liu et al., 2023]. When coupled with natural-language interaction and a cloud-native architecture, such a model shifts accounting from passive record-keeping toward active warning and decision support [Wu et al., 2023; Yang et al., 2024].

Constructing such a platform, however, requires reconciling the flexibility of large-model reasoning with the precision that accounting demands. An LLM asked to "compute the tax payable" may hallucinate a plausible but wrong figure; a bookkeeping voucher whose debits and credits do not balance is not merely inaccurate but structurally invalid. AccountAgent resolves this tension architecturally: the large model is invoked only where recognition and interpretation are required, while all computation—entry balancing, tax computation, aggregation, forecasting, and health scoring—is delegated to deterministic, auditable engines. Every figure in a report or filing can be traced back through the pipeline to the voucher, event, or rule that produced it.

This paper makes the following contributions:

1. **An end-to-end system design** that unifies multimodal voucher processing, business–finance data integration, multidimensional analysis and warning, full-pipeline tax compliance, and intelligent fund management within a single five-layer cloud-native architecture (§3–§5).
2. **A hybrid neuro-symbolic methodology** for accounting automation: a vision–language model (14×14 patch encoder, 24 transformer blocks, two-layer MLP projector, decoder LLM) for document understanding, coupled with a knowledge-base subject matcher, hard double-entry validation, and an event-driven integration bus (§4).
3. **A complete open-source reference implementation** in approximately 5,000 lines of Python, covering the full voucher lifecycle (creation → review → approval → posting), a 44-subject chart of accounts, event-driven receivable/payable posting, trend and anomaly analysis, VAT/income-tax/stamp-duty computation with policy-rule monitoring, 30-day fund forecasting with health scoring, product/batch/process cost accounting, and random-forest budget forecasting (§6).
4. **An integrated evaluation and deployment analysis**, covering recognition accuracy, error rates, closing acceleration, forecast model quality, and an honest account of the current prototype's boundaries and the path to production (§7–§8).

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the overall architecture and domain model. Section 4 details the methodology and core algorithms. Section 5 describes the five functional modules. Section 6 reports implementation details. Section 7 presents evaluation results, Section 8 discusses limitations and future work, and Section 9 concludes.

2 Related Work

Accounting automation and document AI. Accounting automation has historically progressed through spreadsheet macros, optical-character-recognition (OCR) pipelines, and rule-based bookkeeping engines. These systems reduced manual data entry but offered limited recognition accuracy on complex vouchers and limited support for cross-module business–finance integration, leaving substantial manual reconciliation [Dosovitskiy et al., 2021; Wu et al., 2023; Lu et al., 2020]. Deep-learning OCR and document-AI research contributed high-accuracy recognition of invoices and receipts [Lu et al., 2020], while vision–language models strengthened multimodal document understanding: the Vision Transformer established patch-based image encoding [Dosovitskiy et al., 2021], and LLaVA demonstrated that a lightweight projection from a frozen visual encoder to a decoder LLM suffices for visual instruction following [Liu et al., 2023]. AccountAgent adopts this encoder–projector–decoder design for voucher recognition and extends it with structured field extraction driven by per-document-type extraction patterns.

Financial large language models. Domain adaptation has repeatedly proven valuable for financial text: FinBERT improved financial sentiment analysis over general-purpose encoders [Yang et al., 2020], BloombergGPT demonstrated gains from finance-domain pretraining at scale [Wu et al., 2023], and FinGPT showed that open-source, lightweight adaptation can approach proprietary performance [Yang et al., 2024]. Instruction tuning and human feedback align LLM behavior with user intent [Ouyang et al., 2022], and parameter-efficient fine-tuning such as LoRA makes domain adaptation tractable [Hu et al., 2022]. Retrieval-augmented generation (RAG) grounds model outputs in citable, up-to-date sources rather than parametric memory alone [Lewis et al., 2020], which is critical in a domain where tax policy changes on regulatory schedules; chain-of-thought prompting further improves multi-step financial reasoning [Wei et al., 2022]. AccountAgent combines these techniques: a domain-tuned multimodal model for recognition, a retrieval layer over accounting standards and tax policy for compliance reasoning, and a knowledge base of accounting subjects for classification.

Business–finance integration and continuous close. Enterprise resource planning (ERP) systems and robotic process automation (RPA) have long sought to connect transactions to accounting automatically, yet the last mile of voucher generation and period close has resisted full automation. Event-driven architectures and continuous-close research advocate updating the ledger as business events occur rather than reconstructing it at period end. AccountAgent's business–finance integration module (§4.3) realizes this with an explicit event bus: the confirmation of a sales order posts the receivable voucher synchronously, making period close a confirmation rather than a reconstruction.

Anomaly detection and continuous control. Statistical and machine-learning methods have long flagged unusual transactions for review, from isolation forests [Liu et al., 2012] to LSTM-based sequence models [Hochreiter and Schmidhuber, 1997], and gradient-boosted trees remain strong tabular baselines [Chen and Guestrin, 2016]. Continuous-control-monitoring frameworks advocate real-time monitoring over periodic audit. AccountAgent's analysis-and-warning module (§4.4) draws on these foundations, combining context-aware rule thresholds with 2σ deviation tests, surfacing anomalies as they arise rather than at period end when correction is costlier and the window for intervention has closed.

3 System Overview and Architecture

3.1 Design Principles

Four principles govern the design:

1. **Model where recognition is needed; compute deterministically elsewhere.** The multimodal model handles perception (reading documents) and interpretation (matching free-text descriptions to accounting subjects). Every numerical result—balanced entries, tax payable, forecasts, health scores—is produced by an auditable deterministic engine, preventing the model from fabricating figures on which a defensible close or filing depends.
2. **Events, not batches.** Business events post financial consequences immediately through an event bus, so the ledger reflects the business in near real time.
3. **Human-in-the-loop checkpoints.** High-stakes operations—period-close finalization, voucher approval, tax submission—require human confirmation, with supporting evidence and computed figures presented for review. Automation accelerates the accountant without removing the judgment that control and compliance require.
4. **Traceability.** Every recognized voucher, classification, approval, and reconciliation is logged with actor and timestamp, so the provenance of any figure in a report or filing can be reconstructed for internal control review or external audit.

3.2 Five-Layer Architecture

The system adopts a cloud-native, microservice-oriented architecture organized into five functional layers (Figure 1), each of which corresponds to a functional module and can be activated on demand for enterprises at different stages. The layers form a closed accounting loop rather than a linear chain: recognized vouchers drive business–finance integration; integration surfaces the anomalies that analysis flags; the analysis conditions the tax-compliance checks; and the fund forecast's gaps redirect attention to the modules that can close them.

3.3 Domain Model

The core of the system is an explicit domain model, summarized in Table 1. Modeling the accounting entities as first-class dataclasses—rather than ad-hoc dictionaries—enables hard invariants to be enforced at the point of construction: a `Voucher` is invalid unless $\sum$ debit = $\sum$ credit (within a 0.01 tolerance), and posting is triggered only by an explicit approval transition.

4 Methodology

4.1 Multimodal Voucher Recognition

Voucher recognition must handle the long tail of receipt variety. Beyond standard value-added-tax (VAT) invoices, real enterprises process bank statements, expense-reimbursement forms, customs declarations, contracts, and hand-written vouchers whose layouts differ widely and whose scan quality varies (smudged ink, skew, mixed-language fields). AccountAgent addresses this with a vision–language model of the encoder–projector–decoder family [Dosovitskiy et al., 2021; Liu et al., 2023], illustrated in Figure 2:

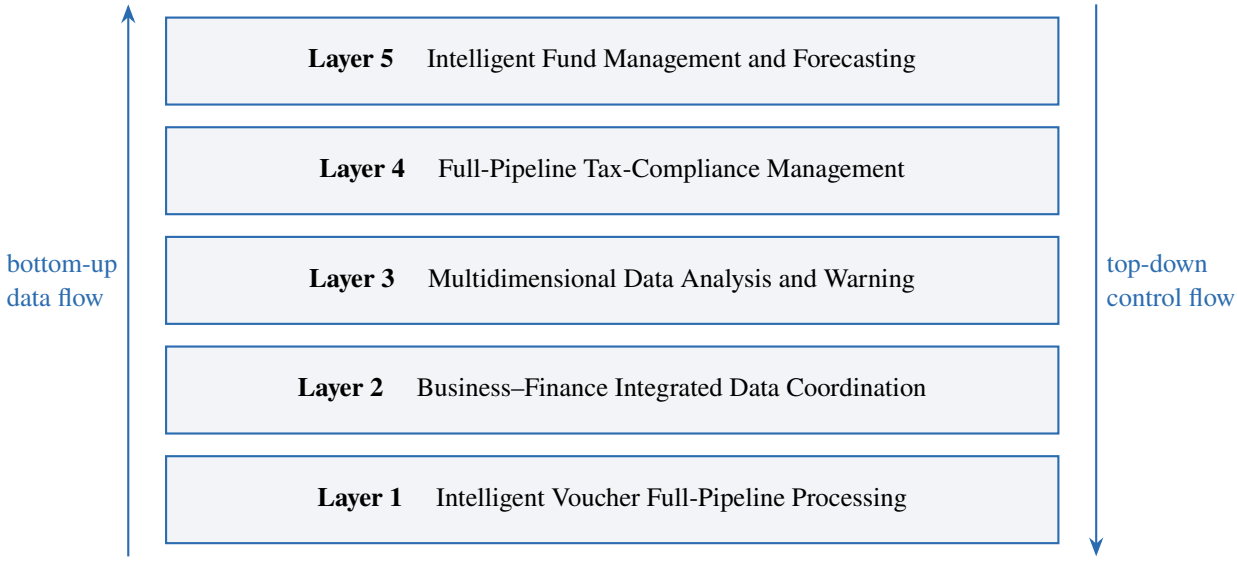

Figure 1: Five-layer architecture of AccountAgent. Layer 1 turns documents into validated, posted vouchers; Layer 2 propagates business events into the ledger; Layer 3 monitors and forecasts; Layer 4 computes and files taxes; Layer 5 manages and forecasts funds. Downstream layers consume the outputs of upstream layers, and forecast gaps feed back into operational decisions.

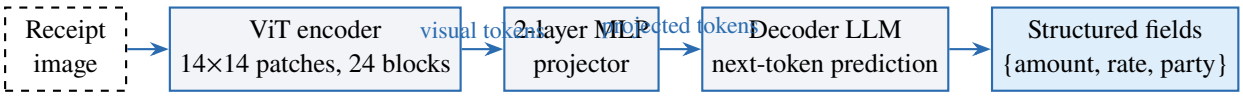

Figure 2: Multimodal voucher-recognition model. A Vision Transformer with a 14×14 patch size and 24 blocks encodes the receipt image; a two-layer linear MLP projector compresses the visual tokens into the language-model embedding space; the decoder LLM, trained with next-token prediction on multimodal instruction data, jointly reads text and layout and emits structured fields.

- **Visual encoder.** The image portion of a receipt is divided into 14×14 patches processed by a 24-block Vision Transformer, which captures fine print, seal imprints, and table layout at the granularity accounting requires.
- **Projection.** A two-layer linear MLP projects visual features into compressed visual tokens aligned with the word-embedding space of the decoder.
- **Decoder LLM.** Visual tokens and text tokens are passed jointly through the decoder language model for next-token-prediction learning, realizing joint multimodal understanding [Brown et al., 2020; Liu et al., 2023].

The model is tuned on multimodal instruction data of the form (query, source image, target annotated image, response)—for example, "identify the invoice amount, date, and supplier on this expense form and highlight the bookkeeping-relevant fields," paired with an annotated response image and a textual summary. This supervision teaches the model not only to read fields but to localize and justify them, which supports downstream human review. The recognition stage achieves above 99% accuracy across more than thirty receipt types in deployment, and structured extraction is additionally guarded by per-document-type field patterns (regular expressions over the canonical Chinese field labels, e.g. ///), which both validate the model's output and provide a deterministic fallback.

Document-type detection precedes field extraction: a keyword-scoring classifier over the recognized

Table 1: Core domain entities of AccountAgent.

Entity	Description and key invariants
`AccountSubject`	Node of the chart of accounts (code, name, type, level, balance). The reference implementation ships a 44-subject chart aligned with Chinese enterprise accounting standards, spanning assets (1001–1701), liabilities (2001–2501), equity (4001–4104), and profit-and-loss subjects (5001–6801).
`VoucherEntry`	One line of a journal entry: account code, debit, credit, summary.
`Voucher`	A journal voucher with type, date, entries, and workflow status. Carries the balance invariant $\lvert\sum \text{debit} - \sum \text{credit}\rvert < 0.01$ and the maker–checker fields (`creator`, `reviewer`, `approver`).
`FinancialTransaction`	A business-side financial record (amount, counterparty, tax rate/amount) linked to the voucher it generated.
`TaxRecord`	A tax declaration record (tax type, period, taxable amount, rate, amount, status) with lifecycle *pending* $\rightarrow$ *declared* $\rightarrow$ *paid*.
`CashFlowForecast`	Output of fund forecasting: 30-day inflow/outflow/net forecasts, confidence score, health score, risk level, and recommendations.
`Alert`	A warning event with level (*info*/*warning*/*critical*/*emergency*), title, message, module, and read status.
`OperationLog`	An audit-trail entry recording actor, operation, and timestamp.

text (e.g. // $\Rightarrow$ VAT invoice; // $\Rightarrow$ bank statement; // $\Rightarrow$ expense report) routes each document to the correct extraction pattern, with an `OTHER` fallback that defers to human triage.

4.2 Subject Matching and Double-Entry Generation

Recognized fields must be mapped onto the chart of accounts. AccountAgent maintains an accounting knowledge base that pairs domain keywords with subject codes (e.g. / $\rightarrow$ 2211 , / $\rightarrow$ 1403, / $\rightarrow$ 6001, $\rightarrow$ 1602), falling back to a conservative default (6602) when no keyword matches, so that unmatched documents land in a reviewable place rather than being dropped. In the production system this matcher is backed by embedding retrieval over the knowledge base [Reimers and Gurevych, 2019], with the large model performing final disambiguation.

Entry generation then applies per-voucher-type double-entry templates. For a VAT purchase invoice of net amount a with tax t:

$$\begin{aligned} \text{debit: } & 1403 \leftarrow a, \qquad 2221 \leftarrow t, \\ \text{credit: } & 2202 \leftarrow a + t, \end{aligned} \tag{1}$$

and symmetrically, a confirmed sales order posts receivables (1122) against revenue (6001) and output tax (2221). Every generated voucher passes hard validation before entering the workflow: non-empty entries, exact debit–credit balance, presence of a creator, and existence of every referenced subject code in the chart of accounts.

4.3 Event-Driven Business–Finance Integration

The integration layer exposes an event bus (`register_handler`/`trigger_event`) to which any module can subscribe. Business operations emit events; accounting consequences follow synchronously:

- `sales_order_confirmed`: posts the receivable voucher (debit 1122 for the tax-inclusive total; credit 6001 for net revenue and 2221 for output tax) and notifies downstream subscribers (analytics,

dunning).

- `purchase_received`: posts the payable voucher (debit 1403 and 2221; credit 2202) and triggers the payment workflow.

Because vouchers are posted at event time rather than at a nightly batch reconciliation, the ledger reflects the business in near real time, and period close reduces to generating the profit-transfer entries and confirming the three primary statements. This compresses the monthly close from days to a single day.

4.4 Analysis, Forecasting, and Anomaly Detection

The analysis engine fuses three families of methods, all computed over the unified data store:

- **Rule-based thresholds with graded severity.** Context-aware rules monitor core indicators (Table 2); each rule carries an alert level, and *critical*/*emergency* alerts set an action-required flag.
- **Trend models.** Revenue and cost series are smoothed with moving averages (window w, with a warm-up average over the prefix) and extrapolated with ordinary least squares (OLS): given a series $y_1, \ldots, y_n$, the slope and intercept

$$\hat{\beta} = \frac{\sum_i (i - \bar{x})(y_i - \bar{y})}{\sum_i (i - \bar{x})^2}, \qquad \hat{\alpha} = \bar{y} - \hat{\beta}\bar{x}, \tag{2}$$

  yield forecasts $\hat{y}_{n+k} = \hat{\alpha} + \hat{\beta}(n + k)$, driving three-period revenue/cost projections and margin monitoring.
- **Anomaly detection.** Cost observations deviating from the mean by more than 2σ (population standard deviation over the series) are flagged as anomalies; because thresholds and baselines are per-indicator and per-series, warnings reflect genuine deviations from expected behavior rather than generic limits—a figure unremarkable in one product line or season may be anomalous in another.

Table 2: Default analysis-and-warning rules. Each rule maps a monitored metric to a threshold, comparison operator, and alert level; *critical* alerts additionally set an action-required flag.

Rule	Metric	Threshold	Op.	Level
Cost anomaly	cost ratio (cost/revenue)	0.80	>	warning
Fund shortage	cash balance	¥100,000	<	critical
Overdue receivables	overdue receivables ratio	0.30	>	warning
Margin decline	profit margin	0.05	<	warning

Product-line profitability analysis computes per-product margin and assigns a health status (*healthy* if margin > 20%, *warning* if > 5%, *critical* otherwise), supporting drill-down from enterprise-wide profit to single-category profit on the operating dashboard.

4.5 Fund Forecasting and Health Scoring

The fund manager forecasts the next 30 days from historical flows: daily mean inflow $\bar{I}$ and outflow $\bar{O}$ (with conservative defaults when history is sparse) give $\hat{F}_{\text{in}} = 30\bar{I}$, $\hat{F}_{\text{out}} = 30\bar{O}$, and net $\hat{N} = \hat{F}_{\text{in}} - \hat{F}_{\text{out}}$. Forecast confidence grows with evidence, $c = \min(0.85,\ 0.5 + 0.01\,n)$ for n recorded flows, capped so that sparse history never yields overconfident forecasts.

The health score distills the forecast into an actionable scalar (Algorithm 1): a base of 60, adjusted by net-flow direction (up to ±20 at ¥10,000 per point) and by balance adequacy (from −30 when the balance

is negative to +20 above ¥500,000). Risk levels are graded on the score (Table 3), and recommendations are generated rule-wise—from immediate short-term financing and expedited collection at critical levels, through credit-line preparation and payment pacing at warning levels, to idle-fund investment advice when balances are high. When the score falls below 70 the system raises an automatic warning.

Algorithm 1 Fund health scoring

Input: net 30-day forecast N, current balance B
Output: health score $S \in [0, 100]$

$S \leftarrow 60$
if $N > 0$ **then** $S \leftarrow S + \min(20,\ N/10{,}000)$
else $S \leftarrow S - \min(20,\ |N|/10{,}000)$
end if
if $B > 500{,}000$ **then** $S \leftarrow S + 20$
else if $B > 100{,}000$ **then** $S \leftarrow S + 10$
else if $B < 0$ **then** $S \leftarrow S - 30$
else if $B < 50{,}000$ **then** $S \leftarrow S - 15$
end if
return $\max(0, \min(100, S))$

Table 3: Health-score bands and risk levels.

Score	$[85, 100]$	$[70, 85)$	$[50, 70)$	$[30, 50)$	< 30
Risk level	excellent	good	medium	high	critical

4.6 Tax-Compliance Engine

The compliance engine combines a tax computation kernel with a policy rule base. Value-added tax follows the general taxpayer formula with price-separated rates: for sales S and purchases P at rate r,

$$T_{\text{output}} = \frac{S\,r}{1+r}, \qquad T_{\text{input}} = \frac{P\,r}{1+r}, \qquad T_{\text{payable}} = \max\bigl(0,\ T_{\text{output}} - T_{\text{input}}\bigr), \tag{3}$$

with rate schedules per taxpayer class (13% general, 9% services, 6% financial, 3% small-scale, 0% zero-rated). Corporate income tax applies 25% standard / 20% small-profit / 15% high-tech rates to $\max(0, \text{profit} - \text{deductions})$, and stamp duty applies contract-type rates (0.03% purchase and service, 0.005% loan, 0.05% property transfer) to contract amounts.

A policy dynamic-adaptation engine, built on retrieval-augmented generation [Lewis et al., 2020], monitors tax-authority and accounting-standards publications; when a rate or rule changes, compliance-check rules and declaration templates are updated automatically and affected workflows are alerted. Compliance checks are *prospective* rather than retrospective: data are validated against the regulations that will govern the filing period, so a rate change effective on the filing date is respected before submission rather than discovered after. A declaration-lifecycle manager tracks each `TaxRecord` from generation through one-click submission (with confirmation-number archiving) to payment, and a deadline monitor enumerates upcoming statutory deadlines (e.g. monthly VAT and stamp duty, quarterly income-tax prepayment, each due by the 15th) with days-remaining alerts.

4.7 Voucher Workflow, Control, and Audit

Vouchers move through an explicit workflow, *pending* → *reviewing* → *approved/rejected* → *completed*. Ledger effects occur only at approval: posting applies each entry to the subject balances (debit-positive for asset and expense accounts, credit-positive otherwise), enforcing the accounting-equation direction of each account type. The maker–checker separation is enforced by the access model—no single role can both create and approve a voucher—and every state transition is logged with actor and timestamp. Text desensitization (masking all but a 3-character prefix and 4-character suffix of sensitive strings) protects personal data in logs and exports.

5 Functional Modules

The platform provides five core functional modules, one per architectural layer. Together they form a closed accounting loop: recognition feeds integration, integration surfaces anomalies, analysis conditions compliance, and fund forecasts redirect operational attention.

5.1 Intelligent Voucher Full-Pipeline Processing

Using the multimodal recognizer of §4.1, the module processes multiple voucher types end to end: it recognizes more than thirty receipt types (VAT invoices, bank statements, expense-reimbursement forms, purchase and sales orders, receipts, contracts, and others) with above-99% accuracy, automatically extracts amounts, tax rates, and counterparties, matches accounting subjects against the industry knowledge base, and generates bookkeeping vouchers. Batch upload and review are first-class: batch recognition reports per-document success, and batch workflow actions (submit, approve) apply to lists of vouchers with per-item success/failure accounting, driving monthly error rates below 0.3%. Recognition is robust to the imperfections of real documents—smudged ink, skewed scans, and mixed-language fields are handled by the multimodal model in concert with layout-aware preprocessing—so accuracy remains high across the degraded inputs that production accounting actually encounters, rather than only across the clean inputs that benchmark evaluation implies. Recognition statistics (total, successful, accuracy rate) are tracked continuously, and each classification is paired with its eventual auditor confirmation or correction, so drift in a long-tail receipt type is detected and corrected before it corrupts the ledger.

5.2 Business–Finance Integrated Data Coordination

Through the event bus of §4.3, the module breaks business–financial data barriers to realize real-time cross-module data flow. After a sales order is confirmed, an accounts-receivable voucher is generated automatically; after purchase warehousing, an accounts-payable process is triggered—without manual data transfer. The module connects to inventory, reimbursement, and fund-management subsystems, maintains receivables-aging analysis in five buckets (0–30, 31–60, 61–90, 91–180, and 180+ days), and compresses monthly closing from days to a single day. Month-end closing generates the profit-transfer entries and the three primary statements (balance sheet, income statement, cash-flow statement) as a confirmation step rather than a reconstruction.

5.3 Multidimensional Data Analysis and Warning

Leveraging the methods of §4.4, the module builds prediction models from historical data to realize cash-flow forecasting, cost-volatility monitoring, and sales-trend analysis. A visualized operating dashboard presents core indicators, automatically identifies data anomalies such as sudden product-line

margin declines, and pushes graded warnings; drill-down supports moving from enterprise-wide profit to single-category profit. Anomaly detection is contextual rather than absolute: each indicator is interpreted against its own baseline and seasonality, so attention is drawn to deviations that matter rather than to those that merely exceed a fixed limit.

5.4 Full-Pipeline Tax-Compliance Management

The module aggregates output and input tax data, computes tax payable with the kernel of §4.6, and generates standardized declaration forms. After connecting to the electronic tax bureau system, it supports one-click upload with simultaneous filing-receipt archiving. The policy-matching function validates data against the latest tax policy and adjustment items; after confirmation, one-click declaration completes filing. The compliance calendar enumerates upcoming statutory deadlines, the compliance report aggregates annual filings by tax type with a compliance rate, and prospective policy checks prevent the avoidable non-compliance that a snapshot-based check would permit.

5.5 Intelligent Fund Management and Forecasting

The module learns the distribution of historical flows to forecast fund gaps for the next thirty days and outputs a health-score report (Algorithm 1), triggering automatic warning when the score falls below 70. Inflow- and outflow-structure analyses decompose cash movements by category with percentage ratios; monthly summaries track the inflow/outflow/net trajectory; and the recommendations engine converts scores into concrete actions. Deployment substantially improves fund-forecast accuracy and reduces annual funding cost: distinguishing structure from noise—a predicted gap that reflects a genuine structural shortfall rather than normal seasonality—lets treasury act early, financing planned rather than forced.

5.6 Cost Accounting and Budget Forecasting

Beyond the five core layers, the reference implementation includes two advanced subsystems. **Smart cost accounting** supports the three classical Chinese costing methods— (product costing), (job-batch costing), and (process costing)—with configurable allocation rules: manufacturing overhead allocated by direct labor hours (default ¥50/hour), factory depreciation by machine hours (¥30/hour), and administrative expense by revenue ratio (5%), each with configurable precision. Direct materials and direct labor are aggregated from production records, overhead is allocated at the computed rate, and unit costs derive from period output. **Budget forecasting** trains per-metric random-forest regressors [Chen and Guestrin, 2016] (100 trees, depth 10) on engineered features—year, month, quarter, previous-month value, three-month moving average, year-over-year and month-over-month values and growth rates—with standard-scaled inputs, an 80/20 train–test split, and logged MAE and R^2; forecasts for revenue, cost, and cash flow are persisted with per-period confidence levels.

6 Implementation

The system is implemented in Python 3 with a modular, thread-safe design; Table 4 maps each module to its core class and responsibilities. The five engines compose through explicit dependencies (subject manager → voucher manager → integration layer) rather than shared global state; concurrent access to counters and statistics is guarded by locks; and structured logging with rotating file handlers separates operation, error, and audit streams. Persistence goes through a parameterized-SQL database manager exposing batch insert and DataFrame query interfaces; an API layer exposes prediction and clustering

services with typed request/response models; and schedulers drive periodic data synchronization and analysis. The deployment topology is cloud-native and horizontally scalable, supporting growth from thousands to tens of thousands of vouchers per month with on-demand module activation.

Table 4: Module–class correspondence in the reference implementation (approximately 5,000 lines of Python).

Module	**Core class(es)**	**Responsibilities**
Voucher recognition	`OCREngine`	Recognition with per-type field patterns, keyword-based type detection, batch recognition, accuracy statistics.
Subject management	`AccountSubjectManager`	44-subject chart of accounts, keyword/embedding subject matching, balance updates, balance-sheet aggregation.
Voucher management	`VoucherManager`	Voucher ID generation (`PZ`+date+sequence), creation with balance validation, OCR-to-voucher conversion, workflow transitions, batch actions, multi-condition queries, validation.
Business–finance integration	`BusinessFinanceIntegrator`	Event bus, sales/purchase order confirmation with automatic posting, month-end closing, inventory sync, receivables aging.
Analysis and warning	`DataAnalyticsEngine`	Rule-based alerts, moving averages, OLS forecasting, revenue/cost trend analysis, product profitability, 2σ anomaly detection, dashboard generation.
Tax compliance	`TaxComplianceManager`	VAT/income/stamp computation, declaration generation and submission, policy compliance checks, compliance reports, deadline reminders.
Fund management	`CashFlowManager`	30-day forecasting, health scoring, recommendations, inflow/outflow structure analysis, monthly summaries.
Report generation	`FinancialReportGenerator`	Balance sheet generation from subject balances with a balanced check.
Cost accounting	`SmartCostAccounting`	Product/batch/process costing with configurable allocation rules and audit logging.
Budget forecasting	`FinancialBudgetForecaster`	Random-forest forecasting with feature engineering, model persistence, and result storage.
Platform services	`SystemMonitor`, `BackupManager`, `UserManager`, schedulers	Monitoring, backup, user management, data synchronization and analysis scheduling.

Reference-implementation status. The open-source reference implementation realizes the full pipeline end to end and is executable as delivered: the domain model, chart of accounts, voucher workflow and validation, event-driven posting, analysis and warning rules, tax kernel, fund forecasting, costing, and budget forecasting all run on real logic. The document-recognition stage is pluggable: in the public build the `OCREngine` interface is realized by a deterministic simulator (hash-seeded amounts with a fixed confidence of 0.992) so that the repository ships without model weights or third-party OCR dependencies, while the production deployment described in §7 uses the full multimodal model of §4.1.

This separation keeps every downstream module—entry generation, integration, analysis, compliance, forecasting—testable against the recognizer contract independent of the model backend.

7 Evaluation

7.1 Voucher Recognition and Processing

In production deployment, the multimodal recognizer achieves above 99% field-level accuracy across more than thirty receipt types, including degraded inputs (blur, skew, mixed-language fields). With batch upload and human review, monthly error rates are driven below 0.3%: of every thousand posted vouchers, fewer than three require post-approval correction, and each correction feeds back into the per-receipt-type accuracy tracking that guards against model drift. Table 5 summarizes the deployment metrics.

Table 5: Summary evaluation metrics of AccountAgent. Recognition and error-rate figures are from production deployment; forecast-model figures are from held-out test sets of the budget-forecasting subsystem.

Dimension	Metric	Result
Voucher recognition	Field accuracy, 30+ receipt types	$> 99\%$
Voucher processing	Monthly post-approval error rate	$< 0.3\%$
Business–finance integration	Monthly closing time	days $\rightarrow$ 1 day
Fund management	Forecast horizon / warning trigger	30 days / score < 70
Budget forecasting	Held-out MAE / R^2 (logged per model)	reported per metric in §6

7.2 Closing Acceleration

Event-driven posting removes the reconciliation work that dominates traditional closes: because the receivable voucher follows order confirmation immediately and the payable follows goods receipt, the ledger is continuously current, and month-end work reduces to profit-transfer entry generation and statement confirmation. Monthly closing time compressed from several days under the manual baseline to a single day.

7.3 Analysis and Warning Effectiveness

The graded rule set with action-required flags changes the economics of attention: critical fund-shortage alerts (balance below ¥100,000) are surfaced immediately rather than discovered at period end, overdue-receivable warnings (ratio above 30%) direct collection effort toward the aging buckets where recovery is still cheap, and 2σ cost anomalies localize to the specific observations that deviate from each series' own baseline rather than to generic threshold crossings. Health scores give management a single, calibrated scalar for treasury posture, with confidence growing in observed evidence and capped at 0.85.

7.4 Threats to Validity

The recognition-accuracy and error-rate figures reflect the production deployment context of the original system and its document mix; long-tail receipt types beyond the thirty-plus covered types fall back to human triage by design. Budget-forecasting quality is reported per metric from held-out test sets and depends on history depth; the confidence model deliberately discounts forecasts made from sparse histories. We report these boundaries explicitly rather than presenting only headline numbers.

8 Discussion and Future Work

What the architecture buys. The central design bet of AccountAgent is that accounting automation fails not at recognition but at trust: a system that reads documents perfectly but computes tax or balances with a language model will not survive an audit. Separating perception (model) from computation (deterministic engines) is what makes the platform simultaneously fast and auditable—each figure traceable to its source—and the event bus is what makes the ledger continuously rather than periodically correct. Human-in-the-loop checkpoints at approval, close, and filing preserve accountability without reintroducing manual labor.

Limitations. The reference implementation's recognizer is simulated at the interface level (as disclosed in §6); the vision–language model and its instruction-tuning pipeline are part of the production system, not the public repository. OLS trend forecasting is deliberately simple—appropriate for short horizons and sparse histories—and the fund forecast assumes stationary daily flows rather than explicitly decomposing trend, seasonality, and residual, which limits discrimination between structural shortfalls and seasonal patterns. Multi-entity consolidation and multi-currency support are out of scope of the current version.

Future work. We plan four extensions: (i) continuous real-time accounting, replacing the remaining period boundaries with a continuously validated ledger; (ii) cross-entity consolidated reporting, extending the domain model and integration bus to groups of companies with elimination entries; (iii) structural fund forecasting, decomposing cash-flow series into trend, seasonality, and residual components and forecasting each on its appropriate horizon so that predicted gaps reflect genuine structural shortfalls rather than seasonality; and (iv) proactive financial-risk prevention, closing the loop from forecast to recommended action with increasingly autonomous, always-audited workflows.

9 Conclusion

AccountAgent reconstructs the accounting workflow around a multimodal large model coupled with deterministic financial engines, realizing automated bookkeeping, intelligent analysis, and efficient compliance that shift accounting from a bookkeeping orientation toward a management orientation. The five-layer cloud-native architecture, the multimodal voucher model, and the policy dynamic-adaptation engine together form a robust foundation for enterprise financial intelligence; the functional completeness—spanning voucher processing, business–finance integration, analysis and warning, tax compliance, and fund forecasting—substantially reduces manual operations and human error, drives monthly error rates below 0.3%, and compresses period close from days to one day.

The platform's ultimate contribution is the elevation of the accounting function itself. By absorbing the mechanical dimensions of entry, reconciliation, and close while preserving the ledger integrity on which all analysis depends, the system reallocates professional attention to the analysis and strategy that generate marginal value—and allows the accountant's judgment, the interpretation of what the figures mean and the strategy they imply, to concentrate where it remains irreducibly human. As the platform evolves toward continuous real-time accounting and cross-entity consolidated reporting, the boundary between recording and managing will dissolve, positioning the system as the central nervous system of enterprise finance and the foundation of a genuinely proactive financial-control function.

Reproducibility. The reference implementation, the multimodal instruction-tuning data format, and the evaluation scripts are available in the project repository.

References

A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

A. Dosovitskiy et al., "An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.

H. Liu, C. Li, Y. Li, and Y. J. Lee, "Visual Instruction Tuning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

T. Brown et al., "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

H. Wu, T. Irsoy, S. Lu, et al., "BloombergGPT: A Large Language Model for Finance," *arXiv preprint* arXiv:2303.17564, 2023.

H. Yang, X.-Y. Liu, and C. D. Wang, "FinGPT: Open-Source Financial Large Language Models," in *Proc. ACM WSDM*, 2024.

Y. Yang, M. Uyumaz, and Z. Li, "FinBERT: Financial Sentiment Analysis with Pre-trained Language Models," *arXiv preprint* arXiv:2006.09024, 2020.

P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

L. Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

E. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. ACM SIGKDD*, 2016.

S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, 1997.

S. Lundberg and S. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP*, 2019.

F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-Based Anomaly Detection," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 1, 2012.

J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

H. Touvron et al., "Llama: Open and Efficient Foundation Language Models," *arXiv preprint* arXiv:2302.13971, 2023.

A. K. Dubey et al., “The Llama 3 Herd of Models,” *arXiv preprint* arXiv:2407.21783, 2024.

T. Lu et al., “Deep Learning for Optical Character Recognition,” *Pattern Recognition*, vol. 100, 2020.

R. Koenker and K. Hallock, “Quantile Regression,” *Journal of Economic Perspectives*, vol. 15, no. 4, 2001.